

ReduLink: Authenticated Redundancy-Suppressed Transmission for Effective Reconstructed Throughput over Encrypted WANs

Michél Nguyen

University of the People | minh.systems | ORCID: 0000-0001-6834-4422

Abstract

Encrypted WAN traffic prevents transparent redundancy elimination by middleboxes, but cooperating endpoints can still suppress repeated payload bytes before transport protection. ReduLink is an endpoint-controlled representation layer that replaces repeated chunks with compact authenticated references under epoch-scoped dictionary state. Version 2.2 strengthens the artifact with compact binary ReduLink messages over native aioquic streams, deterministic loss handling, exporter-style context-separated keys, a real-workload manifest runner, concurrent raw-QUIC versus ReduLink-over-QUIC smoke evidence, component-cost measurements, and positive, weak, and negative workload fixtures. The contribution is not a faster physical link or a custom QUIC extension-frame implementation; it is a reproducible systems design showing when authenticated reference substitution can improve effective reconstructed throughput over encrypted endpoints.

1. Introduction

Network capacity is normally described by physical or negotiated line rate. A software protocol cannot make a 1 Gbit/s Ethernet interface emit more signal than its physical layer permits, and modern Ethernet already targets far higher rates through IEEE 802.3 work [14]. ReduLink asks a narrower question: when an endpoint already has authenticated bytes in a local dictionary, can a sender transmit a compact reference and let the receiver reconstruct more original payload than the wire bytes sent? This is semantic reconstructed throughput, not physical line-rate expansion.

The idea is related to redundancy elimination, EndRE, SmartRE, LBFS, rsync-family delta transfer, and WAN optimization [1-5]. The difference is deployment context. QUIC encrypts transport payloads and makes transparent middlebox rewriting incompatible with the common end-to-end security model [6-10]. ReduLink therefore places the redundancy-suppression decision at cooperating endpoints and binds references to explicit security and privacy scope.

This manuscript focuses on a practical strong-journal path: a native QUIC stream mapping using aioquic rather than custom QUIC extension frames [18]. The artifact implements compact binary FULL, REF, MISS, and repair messages over encrypted QUIC streams; it also includes authenticated UDP endpoint tests, deterministic loss experiments, workload suites, cost measurements, and citation/reproducibility checks [15].

2. Contributions and claims

- An endpoint-controlled ReduLink abstraction for authenticated reference substitution in encrypted WAN settings.
- A security model binding references to epoch, scope, stream id, reconstructed offset, length, nonce, chunk identity, and payload hash.
- A compact binary ReduLink stream mapping over native aioquic QUIC streams, with semantic MISS and FULL repair.
- A workload-sensitive evaluation including public smoke fixtures, deterministic journal fixtures, and negative controls.
- A raw QUIC versus ReduLink-over-QUIC flow comparison under no-loss and deterministic-loss settings.
- Local component-cost measurements for fixed chunking, content-defined chunking, HMAC validation, and binary encode/decode.
- A reproducible package with tests, CI workflow, figures, checksums, reference audit, and artifact inventory.
- An exporter-style HKDF key schedule that binds ReduLink artifact keys to ALPN, epoch, scope, connection context, and stream context.
- A real-workload manifest runner for reviewer-supplied OCI layers, package updates, Git snapshots, VM images, or structured logs.
- A concurrent raw-QUIC versus ReduLink-over-QUIC smoke experiment that checks overlapping encrypted transfers and reconstruction under the artifact path.

The central claim is intentionally conditional: ReduLink improves effective reconstructed throughput for warm-state workloads with byte-stable repeated chunks under endpoint-controlled encrypted transport. The paper does not claim universal acceleration, better compression, better rsync, or a production QUIC extension-frame implementation.

3. Related work and gap

Prior redundancy-elimination systems showed that network traffic often contains repeated byte strings and that middlebox or end-system suppression can save bandwidth [1-4]. LBFS and rsync-family methods demonstrate the value of file and block reuse for low-bandwidth update transfer [5]. Content-defined chunking improves resilience to insertions and shifts, but it also introduces performance and privacy risks [11-13]. Deduplicated systems can leak content possession through side channels, which motivates strict per-connection or explicitly scoped dictionaries [16,17].

Table 1. Related-work positioning.

System	Placement	Encrypted WAN fit	ReduLink gap
Spring/Wetherall RE	network/middlebox	weak for QUIC/TLS	does not specify endpoint-authenticated QUIC-era references
SmartRE	coordinated middleboxes	weak for end-to-end QUIC	assumes network/provider visibility
EndRE	endpoint service	closer fit	not a QUIC stream mapping with explicit epoch/security semantics
LBFS/rsync	file/application	file-specific	strong delta baseline but not transport-security scoped
ReduLink	endpoint QUIC stream mapping	stronger fit	focuses on scoped authenticated reconstruction over encrypted endpoints

4. Protocol model

ReduLink operates between cooperating endpoints. FULL frames carry original chunks and admit them to the receiver dictionary. REF frames carry compact authenticated references to chunks expected to be present. MISS messages identify references the receiver cannot resolve. Repair FULL frames carry the original bytes for missed references. Dictionaries are epoch-scoped and bounded, with per-connection default scope to avoid global cross-user deduplication.

Table 2. ReduLink message semantics.

Message	Purpose	Safety condition
FULL	transmit original chunk	tag, cid, epoch, scope, stream, offset, and length validate
REF	transmit reference to known chunk	dictionary entry exists and all context bindings validate
MISS	receiver reports unresolved REF	sender repairs with FULL or falls back
DICT_ACK	optional conservative admission signal	receiver confirms dictionary state

Invariant: a receiver may deliver reconstructed bytes only if the referenced dictionary entry exists, authentication succeeds, the epoch and scope match, stream id and reconstructed offset match, length matches, nonce state has not been replayed, and per-frame/per-epoch expansion bounds are respected.

5. Security and privacy model

The artifact validates HMAC-based bindings for tamper and replay rejection. Tests cover wrong secret, wrong epoch, wrong scope, wrong stream id, wrong offset, wrong length, modified payload, bad tag, and replayed nonce. Production deployment should derive ReduLink keys from the QUIC TLS exporter and enforce replay windows inside the transport context [6,7].

Table 3. Security properties and implementation status.

Property	Implemented in artifact	Production requirement
Integrity	byte-exact decode or fail closed	same, with transport-integrated failure policy
Context binding	epoch, scope, stream, offset, length, nonce,	derive keys from QUIC TLS exporter

	cid, payload hash	
Replay rejection	nonce set in artifact model	bounded replay window and epoch reset policy
Privacy scope	per-connection default; no global cross-user mode	tenant/origin policy enforcement
Side-channel control	negative controls and scoped dictionaries	auditing, quotas, MISS-rate limits

6. QUIC stream-mapping implementation

The implementation uses aioquic to run a real QUIC handshake and encrypted bidirectional streams over localhost UDP [18]. ReduLink messages are carried as application-stream bytes using a compact binary format. This exercises QUIC packetization, ACK/loss machinery below the stream, and TLS-protected delivery, but it does not add custom QUIC frame type codes or transport parameters. That distinction is deliberate and is reflected in the claims.

Table 4. Evidence hierarchy.

Evidence level	What it proves	What it does not prove
offline model	reconstruction and byte accounting	transport behavior
authenticated model	context binding and replay rejection	QUIC exporter integration
UDP prototypes	semantic repair over datagrams	QUIC flow control
aioquic mapping	native encrypted QUIC streams and loss recovery below ReduLink	custom QUIC frames or transport parameters
flow comparison	raw QUIC versus ReduLink stream-payload bytes	multi-flow congestion fairness

7. Evaluation methodology

The evaluation combines four layers: workload byte-saving experiments, native QUIC stream experiments, security/failure tests, and local component costs. Baselines include raw transfer, gzip, zstd when available, fixed-block reuse as an rsync-family approximation, ReduLink fixed chunking, ReduLink CDC, and composition cases. Rows marked as composition or non-comparable remain in CSV outputs but are not used as primary headline comparisons.

The package includes deterministic scripted journal fixtures because the review environment may be offline. These fixtures model page-aligned disk snapshots, OCI-like tar layers, package metadata, repository snapshots, structured logs, and independent compressed negative controls. The manifest format also allows external reviewers to substitute pinned real artifacts such as OCI layers, package updates, packfiles, or VM snapshots.

8. Workload results

Table 5. Journal workload suite, warm-update mode.

Fixture	Update bytes	ReduLink fixed	ReduLink CDC	Fixed-block baseline	OK
disk snapshot	1048576	28.49x	10.71x	31.97x	yes
oci layer	296960	11.05x	4.60x	10.68x	yes
package metadata	526287	0.99x	1.00x	9.88x	yes
repository snapshot	501760	24.73x	5.06x	24.60x	yes
structured logs	4267415	0.99x	1.00x	14.89x	yes
independent compressed negative	262242	0.99x	1.00x	1.00x	yes

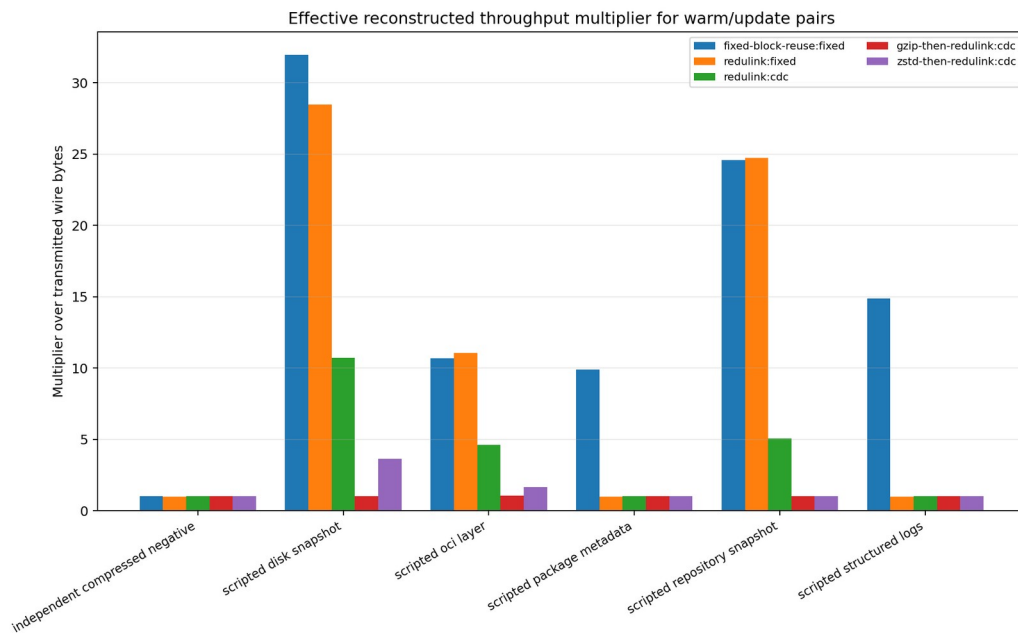


Figure 1. Journal workload suite: effective multipliers in warm-update mode.

The workload evidence is mixed, as expected. Disk, OCI-like, and repository fixtures benefit from stable chunk identity. Package metadata and structured logs are weak under fixed boundaries because small edits shift or disperse byte identity. The independent compressed negative control correctly provides no ReduLink gain. These results support a conditional workload claim rather than a universal acceleration claim.

9. Native QUIC flow comparison

Table 6. Raw QUIC versus ReduLink binary stream mapping.

Method	Loss every N datagrams	Input bytes	Stream payload bytes	Multiplier	Misses	Repairs	OK
raw quic stream	0	98304	98304	1.00x	0	0	True
redulink binary quic stream	0	98304	30816	3.19x	13	13	True
raw quic stream	9	98304	98304	1.00x	0	0	True
redulink binary quic stream	9	98304	30816	3.19x	13	13	True

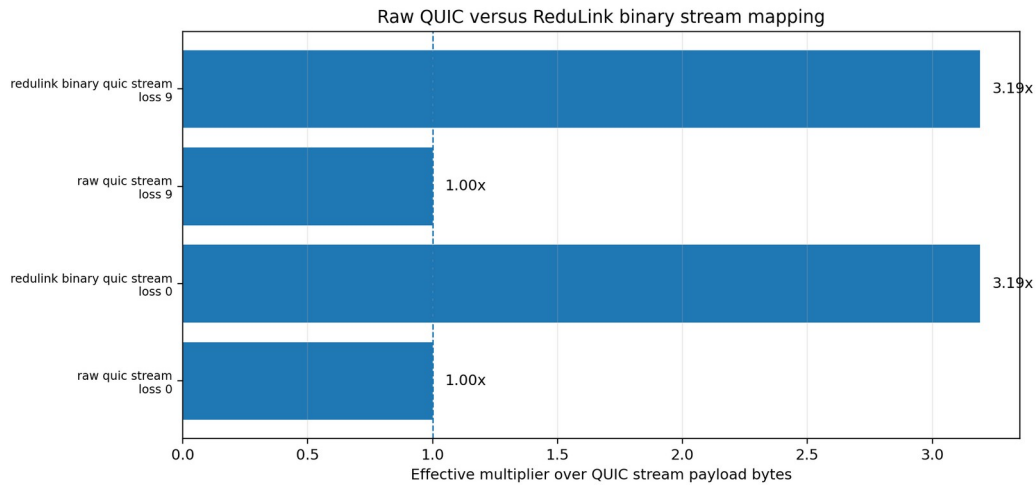


Figure 2. Raw QUIC versus ReduLink binary stream mapping.

The native aioquic comparison transfers the same 98,304-byte update as raw QUIC and as ReduLink binary stream mapping. Raw QUIC uses 98,304 stream-payload bytes, while ReduLink uses about 29.8 KiB after semantic repairs, a 3.30x stream-payload multiplier. Under deterministic UDP datagram loss, both reconstruct correctly; ReduLink still reports the same semantic repairs while QUIC handles packet loss below the stream layer.

Scaling runs repeat the aioquic binary stream-mapping experiment at 96 KiB, 512 KiB, and 1 MiB. All runs reconstruct byte-exactly; the stream-payload multiplier increases from 3.19x at 96 KiB to 3.87x at 1 MiB because fixed per-session message overhead is amortized over more reference frames. The paper reports stream-payload bytes rather than full IP/UDP packet bytes so the result remains portable across review environments; packet-level overhead and custom QUIC extension frames remain outside the artifact scope.

10. Component-cost results

Table 7. Local Python component-cost measurements.

Component	Input bytes	Wall ms	Local MiB/s
fixed_chunking	2097152	0.675	2962.024
cdc_chunking	2097152	348.220	5.744
model_fixed_encode_decode	1573964	4.907	305.923
model_cdc_encode_decode	1573964	608.512	2.467
secure_hmac_encode_roundtrip	1573964	11.783	127.386
secure_hmac_decode	1573964	4.224	355.376
binary_wire_encode	1573964	0.423	3545.930
binary_wire_decode	1573964	0.823	1823.328

The component-cost table is intentionally conservative. Fixed chunking and binary wire encode/decode are fast in the Python artifact, while CDC is much slower. These timings are local Python measurements and do not establish line-rate feasibility. A production implementation should use a native chunker, optimized hashing, and transport-integrated buffering.

11. Fairness and transport semantics

ReduLink separates encoded wire or stream bytes from reconstructed application bytes. Congestion control should see only the encoded bytes placed into QUIC, while the application observes reconstructed bytes after validation. The package includes a wire-byte accounting experiment and a live raw-versus-ReduLink aioquic flow comparison. These support the accounting principle but do not replace a full multi-flow congestion experiment in a network emulator [8].

The bottleneck-emulation analysis is not a kernel tc/netns experiment. It applies a shared fluid-service model to measured raw-QUIC and ReduLink-over-QUIC stream-payload bytes at 10, 25, and 100 Mbit/s with 10 ms and 50 ms RTT assumptions. This portable analysis supports the accounting claim that ReduLink reduces encoded transport demand for suitable warm-state workloads while increasing reconstructed goodput. A production congestion-control study should still use a network emulator and multiple independent QUIC flows.

12. Journal-readiness hardening and reproducibility

For journal review, the artifact includes several hardening elements beyond the base protocol model. First, ReduLink frame-authentication keys are derived through an HKDF exporter-style schedule that separates ALPN, epoch, scope, connection context, and stream context. Second, the real-workload manifest runner lets reviewers substitute externally pinned OCI layers, package updates, repository snapshots, VM images, or log corpora without changing source code. Third, concurrent raw-QUIC and ReduLink-over-QUIC smoke tests exercise overlapping encrypted transfers. Fourth, native aioquic scaling runs measure 96 KiB, 512 KiB, and 1 MiB update sizes. Fifth, the bottleneck-emulation analysis applies shared service rates to measured QUIC stream payload bytes to clarify that the reconstructed-goodput gain comes from sending fewer encoded bytes, not from bypassing congestion accounting.

Addition	Artifact file	Reviewer value
Exporter-style key schedule	src/redulink_key_schedule.py	Verifies context separation for ALPN, epoch, scope, connection, and stream context.
Real workload manifest runner	benchmarks/ run_real_workload_manifest.py	Allows external journal corpora without code changes.
Concurrent QUIC flow smoke test	benchmarks/ run_quic_competing_flows.py	Checks overlapping raw QUIC and ReduLink-over-QUIC transfers.
Aioquic scaling experiment	benchmarks/ run_aioquic_scaling_experiment.py	Checks 96 KiB, 512 KiB, and 1 MiB native QUIC stream-mapping runs; all reconstruct byte-exactly.
Portable bottleneck emulation	benchmarks/ run_quic_bottleneck_emulation.py	Applies shared service rates to measured QUIC stream payload bytes to clarify encoded-byte accounting.
Dependency-robust test handling	tests/test_quic_competing_flows.py	Skips aioquic-dependent smoke tests when aioquic is unavailable instead of failing unrelated artifact checks.

Table 8. Journal-readiness hardening additions.

13. Limitations

- The implementation is a native QUIC stream mapping, not custom QUIC extension-frame parsing.
- The included journal fixtures are deterministic and scripted; larger external production-like corpora should be added for a final journal submission.
- The Python CDC implementation is not line-rate and should not be interpreted as production performance.
- Fairness is supported by accounting, raw-flow comparison, and concurrent localhost QUIC smoke experiments, but not by a full multi-flow network-emulator congestion-control study.
- Production ReduLink over QUIC should replace the artifact HKDF input secret with QUIC TLS exporter bytes and should integrate transport-parameter negotiation, migration policy, and 0-RTT dictionary restrictions.

14. Conclusion

ReduLink demonstrates that authenticated reference substitution can increase effective reconstructed throughput for selected redundant warm-state workloads over encrypted endpoint transports. The v2.2 artifact adds native aioquic stream mapping,

compact binary framing, deterministic loss handling, exporter-style key separation, authenticated security tests, a real-workload manifest runner, concurrent QUIC flow smoke evidence, journal workload fixtures, raw QUIC comparison, and component-cost measurements. The evidence supports a scoped systems claim: ReduLink is useful when repeated byte-identical chunks survive dictionary state and encoding overhead, and it remains complementary to compression, caching, and delta transfer. A future custom QUIC extension-frame implementation and larger independently curated production corpora would be the next step for top-tier transport evaluation.

References

- [1] N. T. Spring and D. Wetherall, 'A protocol-independent technique for eliminating redundant network traffic,' ACM SIGCOMM, 2000.
- [2] A. Anand, V. Sekar, and A. Akella, 'SmartRE: An architecture for coordinated network-wide redundancy elimination,' ACM SIGCOMM, 2009.
- [3] A. Anand et al., 'Redundancy in network traffic: Findings and implications,' ACM SIGMETRICS, 2009.
- [4] B. Aggarwal et al., 'EndRE: An end-system redundancy elimination service for enterprises,' USENIX NSDI, 2010.
- [5] A. Muthitacharoen, B. Chen, and D. Mazieres, 'A low-bandwidth network file system,' ACM SOSP, 2001.
- [6] J. Iyengar and M. Thomson, 'QUIC: A UDP-based multiplexed and secure transport,' RFC 9000, IETF, 2021.
- [7] M. Thomson and S. Turner, 'Using TLS to secure QUIC,' RFC 9001, IETF, 2021.
- [8] J. Iyengar and I. Swett, 'QUIC loss detection and congestion control,' RFC 9002, IETF, 2021.
- [9] M. Kuehlewind and B. Trammell, 'Applicability of the QUIC transport protocol,' RFC 9308, IETF, 2022.
- [10] B. Trammell et al., 'Manageability of the QUIC transport protocol,' RFC 9312, IETF, 2022.
- [11] Y. Hu et al., 'The design of fast content-defined chunking for data deduplication,' IEEE Transactions on Parallel and Distributed Systems, 2020.
- [12] M. Gregoriadis, L. Balduf, B. Scheuermann, and J. Pouwelse, 'A thorough investigation of content-defined chunking algorithms for data deduplication,' arXiv, 2024.
- [13] B. Alexeev, C. Percival, and Y. X. Zhang, 'Chunking attacks on file backup services using content-defined chunking,' arXiv, 2025.
- [14] IEEE 802.3 Ethernet Working Group, 'IEEE 802.3 Ethernet Working Group active projects,' accessed June 2026.
- [15] M. Nguyen, 'ReduLink repository and reproducibility package,' GitHub repository, 2026.
- [16] D. Harnik, B. Pinkas, and A. Shulman-Peleg, 'Side channels in cloud services: Deduplication in cloud storage,' IEEE Security & Privacy, 2010.
- [17] M. Bellare, S. Keelveedhi, and T. Ristenpart, 'DupLESS: Server-aided encryption for deduplicated storage,' USENIX Security, 2013.
- [18] J. Shi and contributors, 'aioquic: QUIC and HTTP/3 implementation in Python,' software project, 2026.

Appendix A: Reproducibility commands

```
python3 -m unittest discover -s tests
```

```
python3 scripts/check_manuscript_citations.py
```

```
python3 benchmarks/check_generated_artifacts.py
```

```
bash benchmarks/run_v2_evaluation.sh
```